MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Report

on laboratory works no. 7 and 8

Operating Systems

Topic: Processes, Redirection and Pipes – from Linux to xv6

Done by:
st. gr. FAF-242, **Vasile Brînză**

Checked by:
Reiţman Patricia, *university assistant*

Chişinău 2026

Table of Contents

PURPOSE OF THE WORK **3**
1 LABORATORY WORK NO. 7 – THREE SMALL PROGRAMS: PROCESSES, REDIRECTION, A PIPE (LINUX) **4**
1.1 Part 1. fork + exec + wait 4
1.2 Part 2. A zombie, on purpose 6
1.3 Part 3. Redirection: ls -l > out.txt 7
1.4 Part 4. A pipe: ls /etc | wc -l 8
2 LABORATORY WORK NO. 8 – THE SAME CALLS, INSIDE XV6 ..... **11**
2.1 Building the new programs 11
2.2 Part 1. run, inside xv6 11
2.3 Part 2. pingpong 14
CONCLUSIONS **16**
REFERENCES **17**

PURPOSE OF THE WORK

The purpose of laboratory works no. 7 and no. 8 is to see in practice how a Unix-like operating system creates and connects processes, first from the outside on a real Linux system, then from the inside, in the small xv6 teaching operating system. By the end of the work the goals are:

- to explain how a process is born and how a program is started, by writing the three steps of the shell – `fork`, `exec` and `wait` – in C and by observing what each of them returns;
- to recognise a zombie process in the process table and to explain which call removes it;
- to implement output redirection (`ls -l > out.txt`) by hand, using the rule that `open()` returns the smallest free file descriptor;
- to build a pipeline (`ls /etc | wc -l`) with `pipe()`, two `fork()` calls and `dup()`, and to explain why every unused end of a pipe must be closed;
- to run the same fork–exec–wait program inside xv6 and to find what is the same and what differs from Linux;
- to write `pingpong` for xv6, where two processes exchange one byte through two pipes.

Laboratory work no. 7 (Track A) was done on a Linux laptop, in bash, as a normal user (`kynex`), in the folder `~/os-lab7`; the four programs were compiled with `gcc -Wall -Wextra`. Laboratory work no. 8 (Track B) was done in the `~/xv6-riscv` tree, compiled with the RISC-V cross compiler and run in the QEMU emulator (`make qemu`). As allowed by the lab assignment, both works are presented in a single report, with a separate chapter for each laboratory work. All outputs shown below are the real outputs of the terminal sessions; the source code and the notes are pushed to the Git repository of the course [1], in the folders `lab7` and `lab8`. The optional tasks of the two labs (`pipeline3.c` and `primes.c`) are not graded and are not part of this report.

1 LABORATORY WORK NO. 7 – THREE SMALL PROGRAMS: PROCESSES, REDIRECTION, A PIPE (LINUX)

The lecture asked three questions: how is a process born, where does `printf` write, and how does `ls | wc -l` work. This chapter answers them with four short C programs. Each part shows the program, the commands with their real outputs, and the answers to the Observe questions of the assignment.

1.1 Part 1. fork + exec + wait

The program `run.c` does what the shell does when the user types `ls -l`: it copies itself (`fork`), the copy becomes `ls` (`exec`), and the original waits for it (`wait`). The three calls behave as follows. `fork()` creates a copy of the calling process and returns twice: the PID of the new child in the parent, 0 in the child and −1 on error. `execlp()` replaces the program of the calling process with another one found through `PATH`; on success it never returns. `waitpid()` blocks the parent until the child has ended and stores the termination status of the child, from which the macro `WEXITSTATUS` extracts the exit code. The two `fflush(stdout)` calls empty the output buffer before `fork()`, so that text already printed is not copied into the child together with the buffer.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command: ls -l */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        printf("child: my PID is %d, my parent is %d\n",
               getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);
    printf("parent: child %d ended with status %d\n",
           pid, WEXITSTATUS(status));
    return 0;
}
```

Listing 1.1 – lab7/run.c

The program was compiled and run three times: first as written (the child becomes `ls -l`), then with the `execlp` line replaced by a call to `date`, and finally with a program that does not exist, `nosuchprog`. After the experiments the original `ls` version was restored.

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 4484
child: my PID is 4485, my parent is 4484
total 24
-rwxrwxr-x 1 kynex kynex 16400 Nov  2 00:03 run
-rw-rw-r-- 1 kynex kynex   688 Nov  2 00:02 run.c
parent: child 4485 ended with status 0
```

Listing 1.2 – run.c with execlp("ls", "ls", "-l", NULL)

```
/* line replaced in the child */
execlp("date", "date", (char *)NULL);

$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 4493
child: my PID is 4494, my parent is 4493
Mon Nov  2 12:03:31 AM EET 2026
parent: child 4494 ended with status 0

/* line replaced in the child */
execlp("nosuchprog", "nosuchprog", (char *)NULL);

$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 4501
child: my PID is 4502, my parent is 4501
exec: No such file or directory
parent: child 4502 ended with status 127
```

Listing 1.3 – The same program with date and with a program that does not exist

The order of the lines is the one expected by the assignment: the parent prints first, then the child prints its own line, then the new program writes its output, and the parent reports the status only after the child has ended.

Answers to the Observe questions

1. The line perror("exec") is in the child, right after execlp. Why is it printed only with nosuchprog?

A successful `execlp()` replaces the whole image of the child (code, data, stack) with the new program, so the instructions that follow the call, `perror("exec")` and `exit(127)`, no longer exist in that process and are never executed. `execlp()` returns only when it fails. With `ls` and `date` the program was found in `PATH` and started; with `nosuchprog` no such file exists, so `execlp()` returned −1, set `errno` to `ENOENT`, and the child continued with `perror("exec")`, which printed `exec: No such file or directory`, and then with `exit(127)` (Listing 1.3).

2. Which number does the child print as "my parent", and whose PID is it?

In the first run the child printed `my parent is 4484`. This is the PID of the original `run` process, the one that printed `parent: my PID is 4484` and called `fork()`; the child obtains it with `getppid()`. The same relation holds in every run: 4493 → 4494 with `date` and 4501 → 4502 with `nosuchprog`.

3. What status does the parent see with ls, and what status with nosuchprog?

With `ls` the parent saw status **0** (and the same with `date`); with `nosuchprog` it saw status **127** (Table 1.1). The number is the value that the child passed to `exit()`: 0 when the program started by `exec` ended

normally, 127 in the child code of `run.c` after a failed `execlp()`. 127 is the same code that bash returns for "command not found".

Table 1.1 – Exit status seen by the parent (WEXITSTATUS)

Program started by the child	Did execlp() return?	Status printed by the parent
`ls -l`	no (the child became ls)	0
`date`	no (the child became date)	0
`nosuchprog`	yes, with −1 and ENOENT	127 (from `exit(127)`)

1.2 Part 2. A zombie, on purpose

A zombie is a child that has ended but has not been waited for. The program `zombie.c` is the original `run.c` (the `ls` version) with one new line, `sleep(20);`, placed just before `waitpid()`, so that the parent sleeps for 20 seconds while the child has already ended (Listing 1.4). The program was started in the background with its output sent to `/dev/null`, and after two seconds the process table was inspected with `ps`. The command was restricted with `-p` and `--ppid` to the parent and its child, so that the line of the shell does not appear (Listing 1.5, Figure 1.1).

```
    int status;
    sleep(20);                              /* the new line */
    waitpid(pid, &status, 0);               /* step 3: wait for the child */
```

Listing 1.4 – The change that turns run.c into zombie.c

```
$ gcc -Wall -Wextra -o zombie zombie.c
$ ./zombie > /dev/null &
$ ZPID=$!
$ sleep 2
$ ps -o pid,ppid,stat,cmd -p "$ZPID" --ppid "$ZPID"
[1] 4520
    PID    PPID STAT CMD
   4520    3471 S    ./zombie
   4522    4520 Z    [ls] <defunct>
```

Listing 1.5 – The process table while the parent sleeps

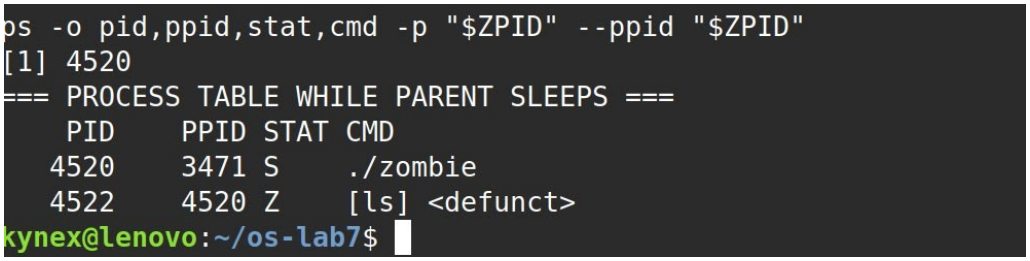


Figure 1.1 – Terminal screenshot: the zombie [ls] <defunct> while the parent sleeps

Table 1.2 – Reading the output of ps

PID	PPID	STAT	Process	Meaning
4520	3471	S	`./zombie`	the parent, sleeping in `sleep(20)`; its own parent, 3471, is the shell
4522	4520	Z	`[ls] <defunct>`	the child: `ls` has finished, but its parent has not called `waitpid()` yet

Answers to the Observe questions

1. Which letter does ls have in the STAT column, and what does it mean?

The letter is **Z**: the process is a zombie (shown as `<defunct>`). The child has terminated and released its memory and open files, but its entry in the process table, with the PID and the exit status, is kept until the parent collects it with `wait()`/`waitpid()`. The name appears in square brackets, `[ls]`, because the command line of a terminated process is no longer available. The `PPID` column (4520) shows that the zombie belongs to the sleeping `zombie` process.

2. After 20 seconds, is the [ls] <defunct> line still there? Which line of zombie.c made it disappear?

No: after the 20 seconds the parent leaves `sleep(20)` and executes `waitpid(pid, &status, 0)`. Because the child has already ended, the call returns immediately, the kernel removes the zombie entry from the process table, and the parent prints its status line and exits. The line that made the zombie disappear is therefore `waitpid(pid, &status, 0);`; the new line `sleep(20);` only delayed it. The terminal session saved for this report contains the first `ps` only (Listing 1.5, Figure 1.1); the disappearance of the zombie after 20 seconds follows from the code and was not captured as a second `ps` output.

1.3 Part 3. Redirection: ls -l > out.txt

The program `redirect.c` does what the shell does for `ls -l > out.txt`. In the child it closes descriptor 1 (standard output), opens `out.txt`, which receives the smallest free descriptor, i.e. 1, and then starts `ls`. The program `ls` is not changed: it still writes to descriptor 1, which now refers to the file (Table 1.3). The parent does not touch its own descriptors, so its message appears on the screen.

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command: ls -l > out.txt */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

Listing 1.6 – lab7/redirect.c

```
$ gcc -Wall -Wextra -o redirect redirect.c && ./redirect
open gave descriptor 1
done: the listing is in out.txt
$ head -3 out.txt
total 68
-rw-r--r-- 1 kynex kynex     0 Nov  2 00:04 out.txt
-rwxrwxr-x 1 kynex kynex 16344 Nov  2 00:04 redirect
```

Listing 1.7 – Running redirect and looking inside out.txt

Table 1.3 – The descriptors of the child in redirect.c

Moment	0 (stdin)	1 (stdout)	2 (stderr)
after `fork()` (copy of the parent)	terminal	terminal	terminal
after `close(1)`	terminal	(free)	terminal
after `open("out.txt", ...)`	terminal	`out.txt`	terminal
after `execlp("ls", ...)`	terminal	`out.txt`	terminal

Answers to the Observe questions

1. Which number did open give, and why exactly that one?

`open()` gave descriptor **1** (the first line of Listing 1.7). The kernel always returns the smallest descriptor that is not in use. The child had 0, 1 and 2 open, and the program closed 1 just before; 0 and 2 were still taken, so 1 was the only free number. The message itself was printed on the screen because it was written with `fprintf(stderr, ...)`, i.e. to descriptor 2, which still referred to the terminal.

2. ls was not changed at all. Why does its listing not appear on the screen?

`ls` writes its listing to descriptor 1, as every program does. The descriptor table of a process is kept by `exec()`, so after `execlp("ls", ...)` the new program inherits the table in which entry 1 points to `out.txt`; `ls` does not know and does not need to know where its standard output goes. The listing therefore went into the file (Listing 1.7), and only the line `done: ...` of the parent, whose descriptor 1 was not changed, appeared on the screen. The file shows `out.txt` with size 0 for a simple reason: the file had been created by `open()`, but `ls` read the sizes before it wrote anything.

3. To redirect the errors instead (like 2> err.txt), which descriptor would you close before open?

Descriptor **2** (standard error). After `close(2)`, the next `open()` would return 2, and everything the program writes to its standard error would go to the file, while the standard output would stay on the screen.

1.4 Part 4. A pipe: ls /etc | wc -l

The program `pipeline.c` builds the pipeline of the shell by hand with one pipe and two children. `pipe(p)` creates a pipe: `p[0]` is its reading end and `p[1]` its writing end. In the first child, descriptor 1 is closed and `dup(p[1])` copies the writing end into the smallest free number, i.e. 1; in the second child the same is done for descriptor 0 and the reading end `p[0]`. After the `dup`, each child closes both original ends and starts its program. The parent also closes both ends and waits for the two children (Table 1.4).

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command: ls /etc | wc -l */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {
        close(1);
        dup(p[1]);
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {
        close(0);
        dup(p[0]);
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p[0]);
    close(p[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

Listing 1.8 – lab7/pipeline.c

```
$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
258
$ echo "Shell result:"
Shell result:
$ ls /etc | wc -l
258
```

Listing 1.9 – The hand-made pipeline compared with the shell

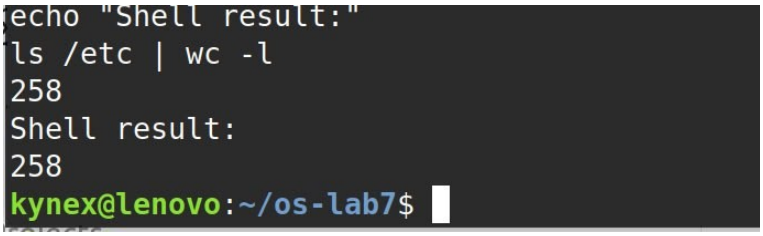


Figure 1.2 – Terminal screenshot: ./pipeline and ls /etc | wc -l both print 258

Table 1.4 – Descriptors of the three processes after the setup of the pipe

Process	0	1	2	Pipe ends held
`ls /etc` (first child)	terminal	**pipe, writing end**	terminal	none besides descriptor 1
`wc -l` (second child)	**pipe, reading end**	terminal	terminal	none besides descriptor 0
parent (`pipeline`)	terminal	terminal	terminal	none: it closed `p[0]` and `p[1]`

To break the program on purpose, the line `close(p[1]);` was turned into a comment and the program was run under `timeout 4`, so that it could not block the terminal. The program printed no line count. After this test the working version was restored and run again; it printed `258` again.

Answers to the Observe questions

1. Did ./pipeline and ls /etc | wc -l give the same number?

Yes. Both printed **258** (Listing 1.9, Figure 1.2), and the restored version printed 258 again. The hand-made pipeline does exactly what the shell does for `|`: one process writes the names of the entries of `/etc` into the pipe and the other counts the lines it reads from it.

2. After the change, which of the two children never ends, ls or wc?

The child that never ends is **wc**. `ls` writes its list and ends normally, as its descriptors are closed when it exits. `wc` stays blocked in `read()` on the pipe, and the parent is blocked too, in the second `wait(NULL)`, because it waits for `wc`; this is why the program does not end and has to be stopped. The bookkeeping of the writing end is in Table 1.5. The terminal session recorded the missing count but no `ps` output, so the identity of the blocked child follows from the mechanism described below.

3. Why, in one sentence: what is wc waiting for?

`wc` waits for the end of file: `read()` on a pipe returns 0 (EOF) only when **all** the writing ends of the pipe have been closed, and the parent still holds `p[1]` open, so the kernel must assume that someone may still write.

Table 1.5 – Who still holds the writing end p[1] after ls has ended

Process	Handout variant (only the parent's close(p[1]) commented)	Variant that was run (every close(p[1]) commented)
`ls`	ended – end closed	ended – end closed
`wc`	closed its own copy	still holds a copy and waits for EOF
parent	**still holds it**, waits for `wc`	**still holds it**, waits for `wc`
result	`wc` never gets EOF – deadlock	`wc` never gets EOF – deadlock

Remark. The editing command used for the test commented out every `close(p[1]);` of the file, not only the one in the parent, which is why Table 1.5 shows both variants. The conclusion is the same in both: as long as one process keeps the writing end open, the reader never receives EOF. Because the parent is the one that keeps it open in both variants, the rule of the assignment is confirmed: **every process must close the ends of the pipe that it does not use**.

2 LABORATORY WORK NO. 8 – THE SAME CALLS, INSIDE XV6

In Lab 7 the calls `fork`, `exec` and `wait` were used on Linux. In xv6 there is no glibc: a user program calls the system calls of the kernel that was compiled in the previous labs, through the small assembly stubs generated from `user/usys.pl` [2]. This chapter runs the same program inside xv6 and then makes two processes talk through pipes (`pingpong`). The sources are in the folder `lab8` of the repository [1].

2.1 Building the new programs

Every new program of xv6 is a file `user/NAME.c` plus one line `$U/_NAME\` in the `UPROGS` list of the Makefile; the line starts with a TAB and nothing may follow the backslash. The two programs of this work were added at the end of the list (Listing 2.1). Before any change, `make qemu` was run to check that the tree builds and boots; the system started the shell as expected.

```
UPROGS=\
       ...
       $U/_sleep\
       $U/_args\
       $U/_find\
       $U/_mystat\
       $U/_run\
       $U/_pingpong\

$ make qemu
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$
```

Listing 2.1 – The end of the UPROGS list and the first boot

2.2 Part 1. run, inside xv6

The program `user/run.c` has the same three steps as the Linux version. The child becomes `echo`, with the arguments `hello from the child`. There is no `execlp`, no `waitpid` and no `fflush` here: xv6 provides `exec(path, argv)`, `wait(&status)` and an unbuffered `printf` [2].

```
#include "kernel/types.h"
#include "user/user.h"

// The three steps of the shell, inside xv6:
// echo hello from the child
int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());

  int pid = fork();
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){
    printf("child: my pid is %d\n", getpid());

    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
```

```
    exec("echo", args);

    printf("exec failed\n");
    exit(1);
  }

  int status;
  wait(&status);
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

Listing 2.2 – lab8/run.c

Three runs were made, each after rebuilding the system with `make qemu`. In the first one the child becomes `echo` (Listing 2.3). In the second one the two lines of the child were changed so that it becomes `ls` (Listing 2.4). In the third one the program does not exist: `exec("nosuchprog", args)` (Listing 2.5).

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
$
```

Listing 2.3 – run with exec("echo", args)

```
/* the two lines that were changed */
char *args[] = { "ls", 0 };
exec("ls", args);

$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36808
echo           2 4 35656
forktest       2 5 17448
grep           2 6 40248
init           2 7 36112
kill           2 8 35600
ln             2 9 35384
ls             2 10 38944
mkdir          2 11 35640
rm             2 12 35624
sh             2 13 58664
stressfs       2 14 52064
usertests      2 15 202520
grind          2 16 52064
wc             2 17 37792
zombie         2 18 34992
logstress      2 19 37576
forphan        2 20 36400
dorphan        2 21 35840
sync           2 22 35056
sleep          2 23 35376
args           2 24 35376
find           2 25 38768
mystat         2 26 36576
run            2 27 35960
console        3 28 0
parent: child 4 ended with status 0
$
```

Listing 2.4 – run with exec("ls", args)

```
/* the changed line */
exec("nosuchprog", args);

$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
$
```

Listing 2.5 – run with a program that does not exist

The PIDs are small because xv6 numbers the processes from 1: `init` is process 1, the shell is process 2, so the first program started from the shell gets 3 and its child gets 4. Each line of Listing 2.4 has the form name, type, inode, size, where type 1 is a directory, 2 a file and 3 a device (the file `console`), as in Lab 6.

Table 2.1 – The same three steps on Linux (Lab 7) and in xv6 (Lab 8)

Step	Linux, run.c of Lab 7	xv6, run.c of Lab 8	What is the same / what differs
1. copy the process	`fork()`	`fork()`	identical: child PID in the parent, 0 in the child, −1 on error
2. become another program	`execlp("ls", "ls", "-l", NULL)`	`exec("echo", args)`	same effect: the image is replaced and the call returns only on error; Linux searches `PATH` and takes the arguments as a list, xv6 takes a path and an array `argv` ended by 0 and has no `PATH`
3. wait for the child	`waitpid(pid, &status, 0)` and `WEXITSTATUS(status)`	`wait(&status)`	same: the parent blocks until the child ends; in xv6 `status` is directly the number given to `exit()`
exit code after a failed exec	`exit(127)`	`exit(1)`	chosen by each program

Answers to the Observe questions

1. Write the three calls that run.c uses in xv6 and the three calls run.c used on Linux in Lab 7. What is the same?

In xv6 the three calls are `fork()`, `exec()` and `wait()`; on Linux in Lab 7 they were `fork()`, `execlp()` and `waitpid()` (Table 2.1). What is the same is the idea and the order of the steps, which is the work of the shell: create a copy, replace the copy by the requested program, wait until it ends. The meaning of the return values is the same too. The interfaces differ only in details: on Linux `execlp()` and `waitpid()` are library functions of glibc built on the system calls `execve` and `wait4`, while xv6 has no such layer and its `exec` and `wait` are the system calls themselves. xv6 has no `PATH`; `exec("echo", args)` works because the shell starts in the root directory, where the file `echo` is [2].

2. Who prints "hello from the child": the code of run, or echo?

The program **echo** prints it. The code of `run` only prints the two lines that begin with `parent:` and `child:`; then `exec("echo", args)` replaces the child by `echo` and passes it the arguments `hello from the`

child, which echo writes to its standard output, i.e. to the terminal inherited from the parent. After a successful exec the rest of the code of run (printf("exec failed")) no longer exists in the child, which is why exec failed is absent from Listing 2.3.

3. Why is the status 1 here and 127 on Linux? (Look at the exit line after exec in both programs.)

The status is not produced by exec and is not fixed by the system: it is the number that the child passes to exit() after exec has failed. The xv6 program has exit(1) after exec failed and the Linux program has exit(127) after perror("exec"). The parent then prints what it receives: 1 in Listing 2.5 and 127 in Listing 1.3. The value 127 was chosen in Lab 7 because it is the convention of the shell for "command not found"; in xv6 the simple value 1 was used.

2.3 Part 2. pingpong

The program user/pingpong.c makes two processes communicate through two pipes. The parent sends one byte to the child through the first pipe (p2c, parent to child). The child reads it, prints its own PID and received ping, writes the byte back through the second pipe (c2p, child to parent) and exits. The parent reads the byte, prints its own PID and received pong, and waits for the child. The ends that a process does not use are closed immediately after fork(). The byte is initialised to 'x' so that a defined value is sent.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];
  char b;
  b = 'x';

  pipe(p2c);
  pipe(c2p);

  if(fork() == 0){
    close(p2c[1]);
    close(c2p[0]);

    read(p2c[0], &b, 1);

    printf("%d: received ping\n", getpid());

    write(c2p[1], &b, 1);

    exit(0);
  }

  close(p2c[0]);
  close(c2p[1]);

  write(p2c[1], &b, 1);

  read(c2p[0], &b, 1);

  printf("%d: received pong\n", getpid());
```

```
  wait(0);
  exit(0);
}
```

Listing 2.6 – lab8/pingpong.c

```
$ pingpong
4: received ping
3: received pong
$
```

Listing 2.7 – Running pingpong in xv6

The child (PID 4) printed `received ping` first, and the parent (PID 3) printed `received pong` after receiving the byte back, as required by the assignment. The sequence of the operations is in Table 2.2.

Table 2.2 – The sequence of pingpong

Step	Process	Operation	Effect
1	parent	`write(p2c[1], &b, 1)`	one byte is in the pipe `p2c`
2	parent	`read(c2p[0], &b, 1)`	the pipe `c2p` is empty: the parent **blocks**
3	child	`read(p2c[0], &b, 1)`	receives the byte (if it ran earlier, it blocked until step 1)
4	child	`printf("%d: received ping\n")`	prints `4: received ping`
5	child	`write(c2p[1], &b, 1)`, `exit(0)`	the byte is in `c2p`; the parent wakes up
6	parent	`printf("%d: received pong\n")`, `wait(0)`	prints `3: received pong`, collects the child, exits

Answers to the Observe questions

1. Why do you need TWO pipes, and not one for both directions?

A pipe is one-directional: what is written into the writing end is read from the reading end, in a single buffer shared by everybody who has the pipe open. With one pipe the parent would write the byte and then read from the same pipe, and it could receive its own byte back before the child had read it; the child would then block forever on an empty pipe, and nothing could tell whose byte is whose. With two pipes each direction has its own buffer: `p2c` carries data only from the parent to the child and `c2p` only from the child to the parent, and each process closes the ends it does not use.

2. Nothing in your code says "wait until the child has replied". Which call makes the parent wait?

The call is the parent's `read(c2p[0], &b, 1)`. When the pipe is empty and its writing end is still open, `read()` does not return 0: it puts the process to sleep until data arrive. The parent therefore waits exactly until the child writes the byte into `c2p`, which also guarantees the order of the output: `received ping` is printed by the child before it writes the reply, and `received pong` can be printed only after the parent has read it. The later `wait(0)` has a different job: it waits for the child process to terminate and collects its status.

CONCLUSIONS

In laboratory work no. 7 the way a Unix shell works was reproduced on Linux with four short C programs. `fork()` creates a copy of the process and returns twice; `execlp()` replaces the copy with another program and returns only when it fails, which is why the line after it runs only for `nosuchprog` and why the exit code seen by the parent was 0 for `ls` and 127 for the missing program. A child that has ended but has not been waited for stays in the process table as a zombie (letter `Z`) until the parent calls `waitpid()`. Redirection needs no change in the program that is started: the descriptor table survives `exec`, so closing descriptor 1 and opening a file makes `open()` return 1 and sends the output of `ls` into `out.txt`. A pipeline uses the same mechanism with `pipe()` and `dup()`; the hand-made `ls /etc | wc -l` printed 258, like the shell, and the test with the writing end left open showed that a reader receives end of file only when every writing end has been closed, so each process must close the pipe ends it does not use.

In laboratory work no. 8 the same three steps were run inside xv6 and showed that the Unix interface is the same in the small teaching system: `fork`, `exec` and `wait` have the same meaning, and the differences are in details such as the missing `PATH`, the argument array of `exec`, and the exit status delivered without `WEXITSTATUS`. The different status after a failed `exec` (1 in xv6, 127 on Linux) is only the choice of the program. The `pingpong` program showed that two pipes are needed for two-way communication and that a blocking `read()` is itself a synchronisation tool: the parent waits for the reply without any explicit waiting code, and the order of the two messages is fixed. The patterns `close(1); dup(p[1])` and `close(fd); open(...)` studied here are the ones used by the shell to implement `|` and `>`, so the programs of both laboratory works explain how the command line of the shell is executed.

REFERENCES

1. Vasile Brînză, GitHub repository. [Online]. Available: https://github.com/kynexi/os-2026-VasileBrinza

2. MIT, "6.1810: Operating System Engineering" (xv6 book and source code). [Online]. Available: https://pdos.csail.mit.edu/6.1810/